

The decryption-oracle question, resolved

For Halil, 2026-08-29

Verdict

The concern applies, and your objection does not dispose of it, though it was the right question to ask. My earlier account of the mechanism was **understated**, and I am correcting it. The attack is not a statistical one that needs many samples. It is one-shot linear algebra.

Two findings are worse than what I told you, and both are verified against our own files rather than asserted.

The smudging parameter in our implementation is **smaller than the noise it is supposed to hide**, by about a factor of 2^{10} . The requirement stated by the multiparty CKKS paper we already cite is larger than ours by about 2^{74} .

Our serving benchmark computes the **maximum logit value**, not the argmax index the method specifies. Neither Go file produces an index.

Neither finding says the protocol design is wrong. Both say that what we measured and what we assumed do not yet match what we specify.

Your question first: does the clients holding the key dispose of it?

No, and the reason is worth stating precisely, because it also tells us who the adversary is.

You are right that a client reading a label is not a confidentiality breach. The clients hold the key by construction and a quorum decrypts by design. Nothing about that is a leak.

The adversary that matters is the one in Theorem 1 and Theorem 2, which is the server together with at most $t - 1$ clients. That coalition **cannot decrypt**. It nevertheless *receives decryption outputs*, because it asks queries through its own corrupted clients and because the selection step announces a decrypted scalar. So the coalition sits in exactly the position the security notion describes, holding honest inputs it did not choose, knowing the circuit, and observing decryptions.

The danger is not that it reads a label. It is that reading a decryption of an **approximate** scheme hands it the noise term, and the noise term is a linear function of the secret key. If the coalition recovers the joint key it decrypts $\text{Enc}(\theta^*)$ directly, and the claim that the model is never disclosed fails cryptographically rather than by extraction. That is a strictly worse failure than anything the query allowance prices.

What I got wrong

I described this as collecting enough noise samples to solve for the key. That overstates the attacker’s cost.

CKKS decryption returns $m' = m + e$. The adversary knows the ciphertext (c_0, c_1) and, in the game, knows m . The relation

$$c_0 + c_1 s = \Delta m + e$$

holds **exactly** in the ring. Knowing m' gives e , and one ring inversion then gives s . A single decryption of a ciphertext whose plaintext the adversary knows is enough. Li and Micciancio report complete key recovery at modest running times.

For our threshold setting the target is the sum of the honest clients’ key shares rather than the joint key directly, because the coalition adds its own shares afterwards. The conclusion is the same.

Is the notion real, and is it defined by someone

Yes. Baiyu Li and Daniele Micciancio, *On the Security of Homomorphic Encryption on Approximate Numbers*, EUROCRYPT 2021, define IND-CPA^D , which is IND-CPA extended with a decryption oracle restricted to honestly evaluated ciphertexts. They prove that for **exact** schemes the two notions coincide, and that for approximate schemes they separate. That separation is the entire point and it is why CKKS needs the stronger notion while BFV and BGV do not.

For the threshold case specifically there is a paper directly on point. Checri, Sirdey, Boudguiga and Bultel, CRYPTO 2024, state in their abstract that existing threshold variants of BFV, BGV and CKKS “would be CPA^D -insecure without smudging noise addition after partial decryption”. So the threshold setting is not an afterthought in this literature. It is called out.

The parameter finding, which is the serious one

What the paper we cite already requires

Mouchet, Troncoso-Pastoriza, Bossuat and Hubaux, PoPETs 2021, is `mouchet2021multiparty` in our bibliography and is the scheme we build on. Their Section 4.5 states the required smudging magnitude:

$$\sigma_{\text{smg}}^2 = 2^\lambda \sigma_{\text{ct}}^2, \quad \text{that is} \quad \sigma_{\text{smg}} = 2^{\lambda/2} \sigma_{\text{ct}},$$

where σ_{ct} is the noise of the ciphertext being switched and λ the security level. Their own security proof leans on it. The key-switch simulator is correct only because the ratio $\sigma_{\text{ct}}^2 / \sigma_{\text{smg}}^2$ is negligible, which is what lets the simulated share be statistically indistinguishable from the real one.

That simulator is precisely the one our Theorem 1 proof invokes when it says the honest clients’ key-switch shares are simulated.

They also flag the motive in their own words, that the key-switch gives the output-key owner the ciphertext noise, which “could be exploited as a side-channel by curious receivers”, and they call characterising it an open question. Li and Micciancio closed it, in the negative.

What we implement

Four sites set the smudging: `fhe/main.go:337`, `fhe/serve.go:193`, and `fhe/protocol_cost.go` at lines 216 and 471. All four use eight times the fresh-encryption noise. With Lattigo’s default of 3.2 that is $\sigma_{\text{smg}} = 25.6 \approx 2^{4.7}$.

The source comment attributes the choice to the library’s own multiparty tests, and that attribution is accurate. It is a test constant chosen so the tests pass. It is not a security parameter.

What our own measurements say the noise is

From `results/fhe_serve/serve_primitives.csv`, at ring degree 2^{15} and one hundred classes, the relative error after a collective refresh is 4.08×10^{-10} , 6.40×10^{-10} and 9.37×10^{-10} at five, ten and twenty parties. At a plaintext scale of $\Delta = 2^{45}$ that puts the ciphertext noise at

$$\sigma_{\text{ct}} \approx 2^{13.8} \text{ to } 2^{15.0}.$$

The gap

quantity	value
implemented σ_{smg}	$2^{4.7}$
our measured σ_{ct}	$\approx 2^{15}$
requirement of the paper we cite, at $\lambda = 128$	2^{79}
shortfall	$\approx 2^{74}$

The implemented flooding noise is about a thousand times **smaller** than the noise it is meant to conceal. It does not conceal it.

The part that has no easy fix

The plaintext scale is $\Delta = 2^{45}$ and the requirement is 2^{79} . Flooding to the stated requirement would exceed the message scale by thirty-four bits and destroy the answer. **The rule our own cited scheme states is not satisfiable at our current parameters.** Satisfying it needs a larger scale and modulus, which costs ring degree, which costs latency and traffic on every number we report.

There is one direction that may be cheaper. A 2025 result claims that in regimes where rescaling noise dominates, a precision loss of as little as two bits restores security against passive key recovery. It is single-key and does not treat the threshold case, so whether it survives the release of partial decryption shares is the one literature question worth resolving before we write anything. I have not verified its body, only its abstract.

The second finding, in our own code

The method specifies, in `alg:serve` line 4, that the server reduces the encrypted logits to $\text{Enc}(\hat{y})$ with $\hat{y} = \arg \max_c \ell_c$, an index.

Both serving benchmarks compute something else. `fhe/serve_tournament.go` runs a rotate-and-Max tournament so that slot zero ends holding the **maximum logit value**, and verifies it against `trueMax`, the largest plaintext logit. `fhe/serve_argmax.go` does the same with a sequential fold, and its own comment reads “verify: decrypted max value matches the plaintext max”. I searched every Go file and none computes an index or a one-hot.

Three consequences.

The reported latency is a **lower bound** for the specified circuit. Turning a max into an index costs a further comparison against each logit, which is C more comparisons and more depth. That compounds with the level-restoration mismatch I corrected yesterday, which was already a lower bound in the other direction.

The claim that the encrypted argmax is exact, with maximum absolute error zero, is a statement about the **max**, not about the index.

Most importantly for the story, the introduction says only a label leaves the protocol, so a client cannot collect the per-class scores that would let it solve for the head. That claim is about the **specified** protocol and it is sound. It is not what the benchmark evaluated. If the served object were the max logit, the querier would receive a real number every query, and the extraction analysis that prices label-only at three to five queries per parameter against seven hundred and sixty-nine for logits would not apply to it.

The fix is to implement the index step, not to weaken the analysis. The specification is the thing that is right here.

A smaller point that follows

The querier decrypts with its own key, because the quorum key-switches to it. So whatever the plaintext is, the querier receives it as a decoded real number and performs any rounding to an index itself. There is no party after the querier to enforce the rounding. Against the server and against non-querying clients the label-only claim holds. Against a corrupted querier, which is the adversary in Theorem 1, the interface does not by itself restrict what is seen to an index.

This does not break the extraction argument, because a corrupted querier that reads the noise still has to do something with it, and what it can do is the key recovery above rather than cheaper extraction. It does mean the sentence should not be read as a cryptographic restriction.

What I recommend, and what I did not do

I have changed nothing in either document. This is a security claim and the decision is yours, and it is now a larger decision than the two clauses I proposed yesterday.

The minimal honest position, if you want the paper correct without new experiments, has three parts.

State the assumption. Theorem 1 should assume the scheme is IND-CPA^D against an adversary holding $t - 1$ shares, and should state the flooding condition the key-switch simulator needs.

Add one step to the proof, covering the released decryptions, saying that the flooding makes the switched noise independent of the honest shares so that the key-switch simulator applies.

Record the limitation honestly. The flooding condition is an assumption on parameters and not a property of the configuration we measured. Anything shorter than that is a claim we cannot support.

Two citations enter, Li and Micciancio for the notion, and Mouchet for the condition, which is already in the bibliography.

If instead you want the paper to *hold* rather than to *assume*, that is a parameter change and a re-measurement, and it will move the latency and traffic figures. That is a bigger decision and it is worth taking deliberately.

The index step is separate and I think it should be done regardless, because it is the gap between what we specify and what we measured, and a reviewer who opens the code will find it.